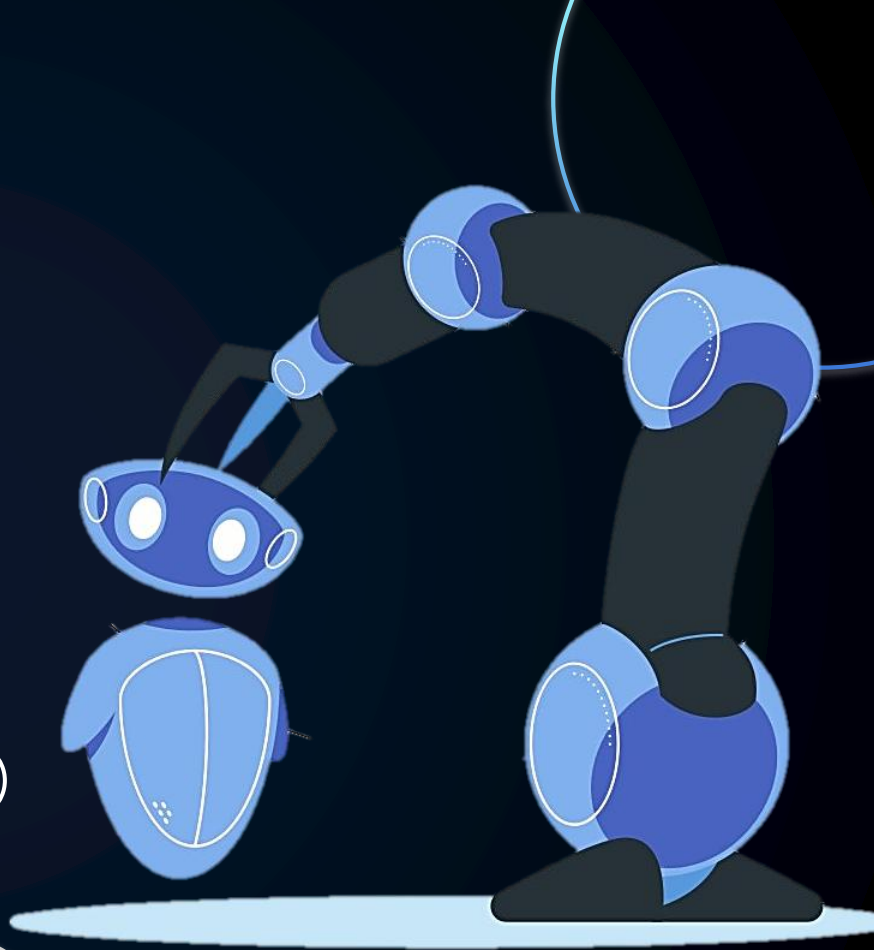
Implicit Human Feedback for Robot Error Correction:

Gaze-Based Interaction in a Tangram Puzzle Task

Heera Menon | Anna Mathew | Susanna Mardbrant



what we have today....

01

Introduction & Base Paper

02

Probem Statement & Our System

03

Build: Ideation, Implementation & Setbacks

04

Results & Lessons

INTRODUCTION

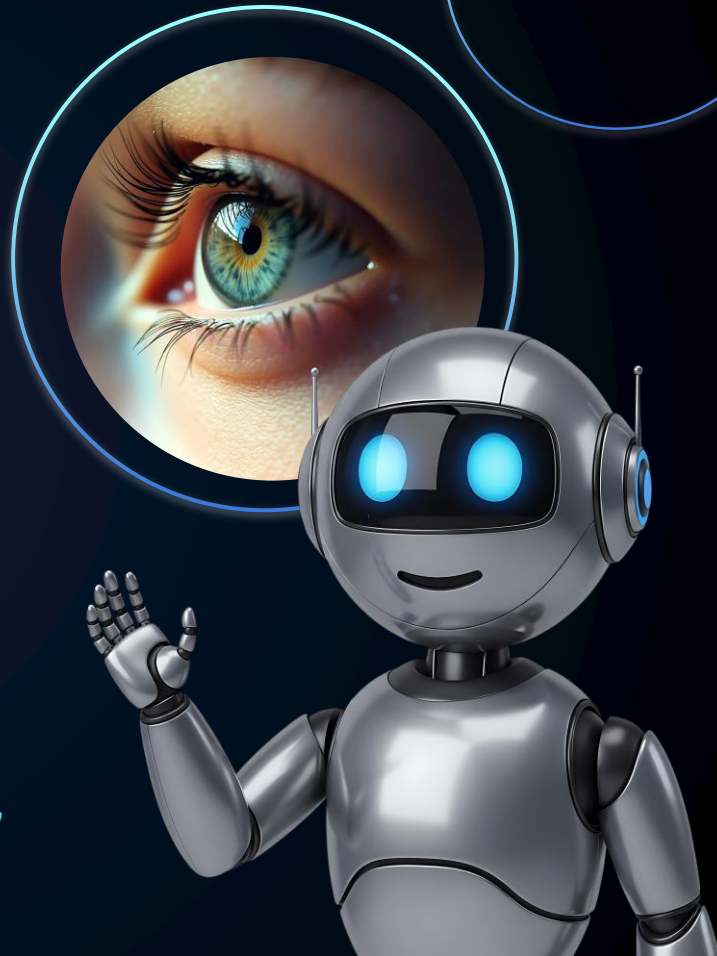
HRI and communication

To achieve efficient human–robot collaboration, robots need to be able to understand human communication cues efficiently. Much of our expression happens without words, and **gaze** is one of the strongest of these implicit signals.

Making use of implicit cues

By using these subtle gaze cues, robots can work with people in a way that Feels smoother and more intuitive. This reduces the need for constant instructions and brings human-robot communication closer to how humans already communicate.

This idea is at the core of our project: enabling robots to understand human reactions through gaze so collaboration becomes easier, more responsive, and more human-like.



so one day we came across….

Gazing at Failure: Investigating Human Gaze in Response to Robot Failure in Collaborative Tasks

Ramtin Tabatabaei
The University of Melbourne
Melbourne, Australia
stabatabaeim@student.unimelb.edu.au

Vassilis Kostakos
The University of Melbourne
Melbourne, Australia
vassilis.kostakos@unimelb.edu.au

Wafa Johal
The University of Melbourne
Melbourne, Australia
wafa.johal@unimelb.edu.au

***Abstract*—Robots are prone to making errors, which can negatively impact their credibility as teammates during collaborative tasks with human users. Detecting and recovering from these failures is crucial for maintaining effective level of trust from users. However, robots may fail without being aware of it. One way to detect such failures could be by analysing humans' non-verbal behaviours and reactions to failures. This study investigates how human gaze dynamics can signal a robot's failure and examines how different types of failures affect people's perception of robot. We conducted a user study with 27 participants collaborating with a robotic mobile manipulator to solve tangram puzzles. The robot was programmed to experience two types of failures —executional and decisional— occurring either at the beginning or end of the task, with or without acknowledgement of the failure. Our findings reveal that the type and timing of the robot's failure significantly affect participants' gaze behaviour and perception of the robot. Specifically, executional failures led to more gaze shifts and increased focus on the robot, while decisional failures resulted in lower entropy in gaze transitions among areas of interest, particularly when the failure occurred at the end of the task. These results highlight that gaze can serve as a reliable indicator of robot failures and their types, and could also be used to predict the appropriate recovery actions.**

***Index Terms*—Robot Failures, Gaze Dynamics, Human-Robot Collaboration**

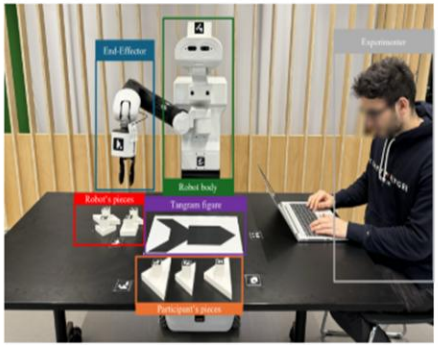


Fig. 1: Different areas of interest in the experiment.

'a fault confessed is half forgiven.' By focusing on moments when human interactions deviate from expected patterns, strategies

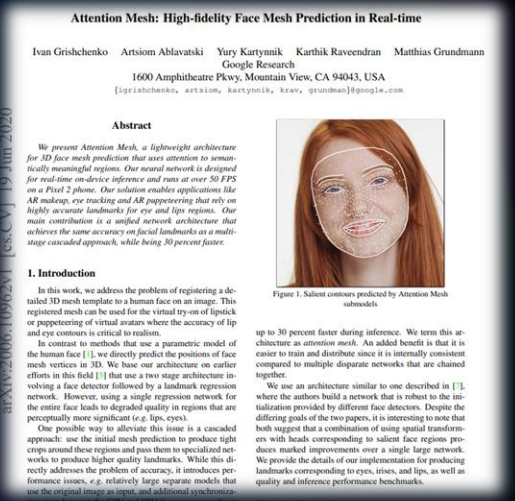
- A physical robot solving a tangram puzzle along with a human participant taking alternative turns
- Analytical study of how well the robot gets the human's view based on multiple matrices of gaze i.e. blinks, dwell times, etc.
- Data was captured using Neon Eye Tracking Glasses from Pupil Labs.
- Insights into using gaze as a failure indicator
- Used the results as inspiration for our metrics

"This finding is similar to the results of Kontogiorgos et al. [29], who found that people tend to gaze more at the robot when it makes a mistake."

REFERENCES

Inspiration for Mediapipes

- They used 468 facial landmarks for facial detection
- They have 5 points out these for eye and iris positioning – which we used!
- So now we get a rough gaze estimation using them
- We used it because:
  - It is lightweight
  - Hardware-independent
  - Compatible with webcams in laptops



Attention Mesh: High-fidelity Face Mesh Prediction in Real-time

Ivan Grishchenko Artsiom Ablavatski Yury Kartynnik Karthik Raveendran Matthias Grundmann
Google Research
1600 Amphitheatre Pkwy, Mountain View, CA 94043, USA
{igrishchenko, artsiom, kartynnik, krav, grundman}@google.com

Abstract

We present Attention Mesh, a lightweight architecture for 3D face mesh prediction that uses attention to semantically meaningful regions. Our neural network is designed for real-time on-device inference and runs at over 50 FPS on a Pixel 2 phone. Our solution enables applications like AR makeup, eye tracking and AR puppeteering that rely on highly accurate landmarks for eye and lips regions. Our main contribution is a unified network architecture that achieves the same accuracy on facial landmarks as a multi-stage cascaded approach, while being 30 percent faster.

1. Introduction

In this work, we address the problem of registering a detailed 3D mesh template to a human face on an image. This registered mesh can be used for the virtual try-on of lipstick or puppeteering of virtual avatars where the accuracy of lip and eye contours is critical to realism.

In contrast to methods that use a parametric model of the human face [1], we directly predict the positions of face mesh vertices in 3D. We base our architecture on earlier efforts in this field [5] that use a two stage architecture involving a face detector followed by a landmark regression network. However, using a single regression network for the entire face leads to degraded quality in regions that are perceptually more significant (*e.g.* lips, eyes).

One possible way to alleviate this issue is a cascaded approach: use the initial mesh prediction to produce tight crops around these regions and pass them to specialized networks to produce higher quality landmarks. While this directly addresses the problem of accuracy, it introduces performance issues, *e.g.* relatively large separate models that use the original image as input, and additional synchroniza-

Figure 1. Salient contours predicted by Attention Mesh submodels

up to 30 percent faster during inference. We term this architecture as *attention mesh*. An added benefit is that it is easier to train and distribute since it is internally consistent compared to multiple disparate networks that are chained together.

We use an architecture similar to one described in [7], where the authors build a network that is robust to the initialization provided by different face detectors. Despite the differing goals of the two papers, it is interesting to note that both suggest that a combination of using spatial transformers with heads corresponding to salient face regions produces marked improvements over a single large network. We provide the details of our implementation for producing landmarks corresponding to eyes, irises, and lips, as well as quality and inference performance benchmarks.

Cyberbotics

Link: *https://cyberbotics.com/doc/guide/tutorials*

Setting up a Webots environment and apply controllers

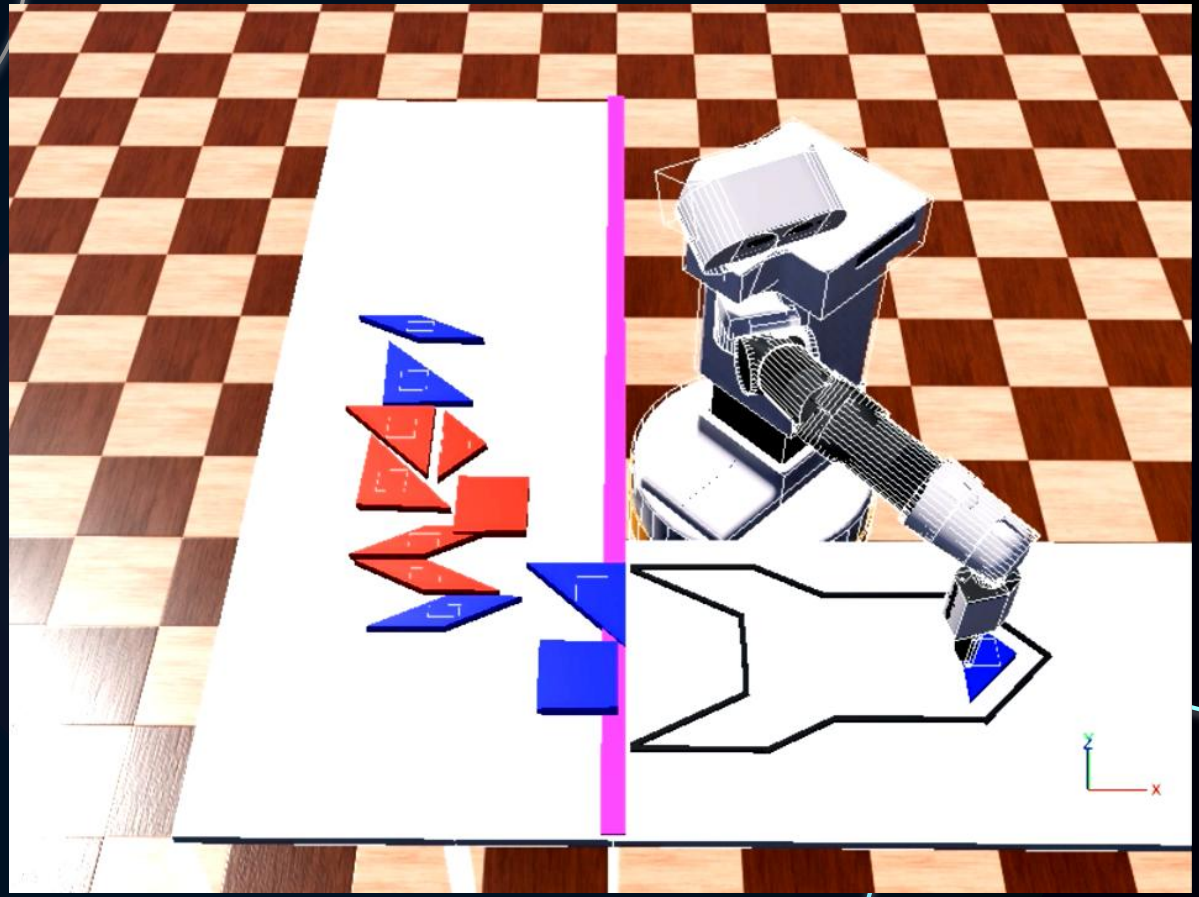
- How Webots worlds are structured (robots, objects, physics settings).
- How to place objects, assign physics properties, and configure simulation timing.
- How to attach a controller to a robot, retrieve devices, and use the simulation step loop.
- How to access and change object properties

So from here we formed....

The Problem Statement

PQ1: *Can a robot take correct decisions only based on implicit human feedback rather than explicit instructions?*

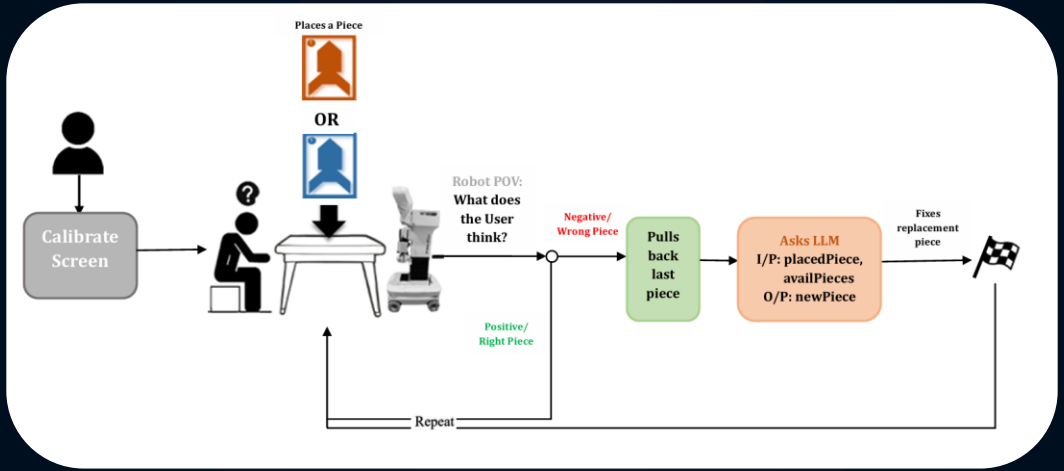
PQ2: *Can a very lightweight, hardware-independent system perform human feedback evaluation? (like gaze through webcams)*



This is just our setup

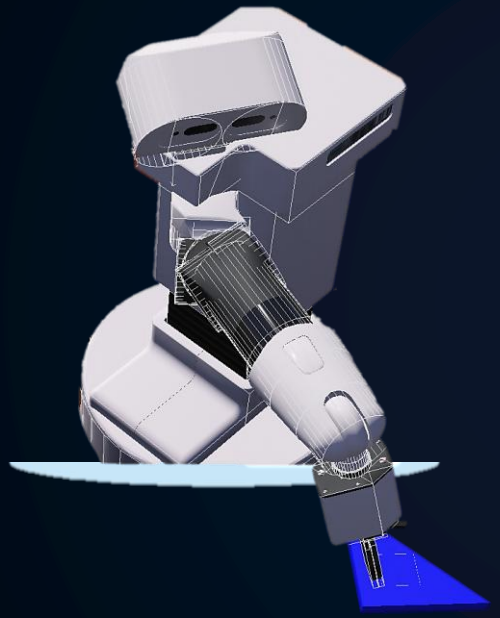
Let's see how we got here & what it does!

Our System & Tasks



We created the adaptation of the setup in the previous paper with a simulated TIAGO robot solving a Tangram Puzzle with a human user.

GAME GOAL: There are two sets of the same puzzle – **RED** and **BLUE** scattered. The user checks the robot into making a puzzle of only one color.



03

THE BUILD

IMPLEMENTATION & SETBACKS


COMPONENTS
WeBots
Gaze Tracking & Algorithm
LLM Integration

WeBots

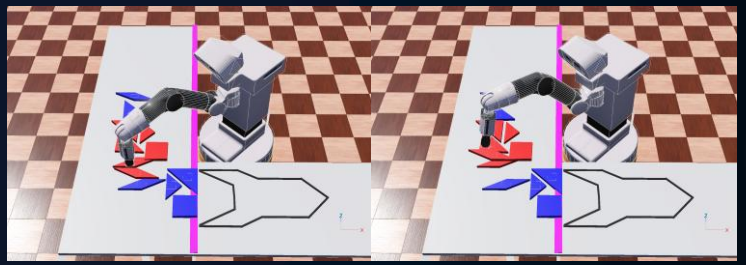
ROBOT KINEMATICS

https://gist.github.com/ItsMichal/4a8fcb330d04f2ccba582286344dd9a

- *IKPY library*
- *Define and build a robot's kinematic chain*
- *Enable, disable, and initialize motors and sensors*
- *Translate world-coordinates into robot-coordinates*

COLLISION AVOIDANCE AND CREATE LEGIBLE MOTION

- Dividing motion into four phases
- Lifting to a safe height, moving horizontally above the table, and descending to grasp or place
- Use tailored IK settings for different error tolerances and optional orientation constraints for each phase



SAFE MECHANISMS

- Detecting when the arm is stuck
- Trigger predefined rescue poses

GRIPPING AND DROPPING PUZZLE PIECES

- simulate “holding objects” by updating the objects translation to follow the robots end-effector
- keep track of the held object by variable “held_piece”

GAME FLOW CONTROL AND INTERACTION HANDLING

- State-handling of which pieces are placed, types already completed, and which pieces are valid next choices based on filtering scattered pieces and excluding types already used.
- Game-logic for coordinating robot actions, including selecting a piece, executing an arm-placement task, starting the gaze-tracking window and removal-sequence.

How Gaze Detection works?

I *Screen Calibrate*

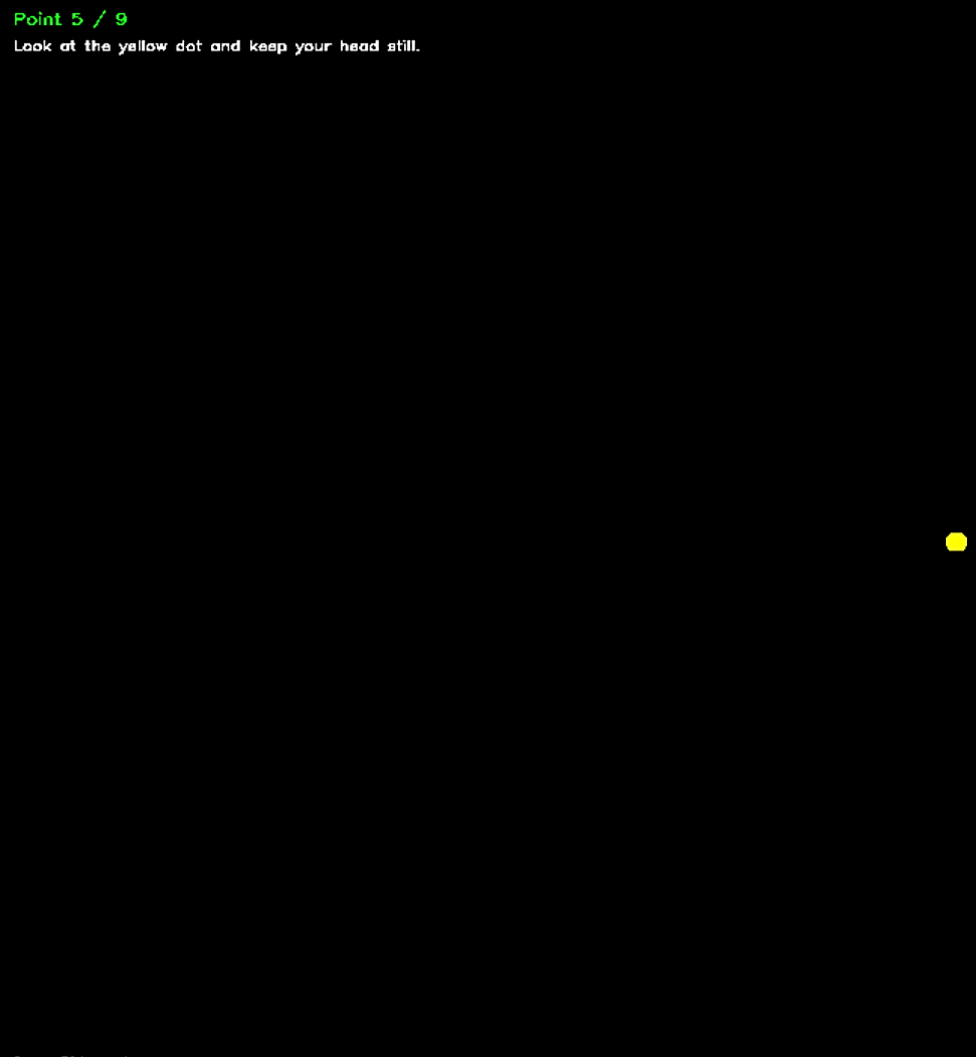
Calibrates that particular user's screen dimensions based on a 9-pointer system to make it compatible for the project.

II *Gaze Detection using MediaPipe*

- Uses MediaPipe points for relatively accurate detection of eye direction.
- When WeBot calls the server, our system using OpenCV and MediaPipe collects user frames for 5 sec and sends it to our metric system to evaluate and conclude the user sentiment.

III *Connection with WeBots*

Connected gaze server to WeBots using HTTP POST request server calls such that after each piece is put down, WeBot calls the server to monitor the user's reaction to every play for 5 sec each.



calibrategaze.py

- ✓ 9 point calibration of top-left, top-centre, top-right, mid-left, centre, mid-right, bottom-left, bottom-centre, bottom-right.
- ✓ Opens up this screen to follow the yellow dots and saves the orientation of your gaze on screen.
- ✓ Each dot halts for 2 sec.

```
(venv) PS C:\Users\HM\Desktop\HRI-project\Webots\gaze_algo> python calibrategaze.py
  warnings.warn('SymbolDatabase.GetPrototype() is deprecated. Please '
2025-12-07 23:03:38,259 INFO __main__: Collected 44 feature samples for point 1
2025-12-07 23:03:38,259 INFO __main__: Point 2 / 9: (1280, 249) — look at the yellow dot.
2025-12-07 23:03:39,066 INFO __main__: Collecting samples for point: (1280, 249)
2025-12-07 23:03:41,095 INFO __main__: Collected 60 feature samples for point 2
2025-12-07 23:03:41,096 INFO __main__: Point 3 / 9: (2176, 249) — look at the yellow dot.
2025-12-07 23:03:41,907 INFO __main__: Collecting samples for point: (2176, 249)
2025-12-07 23:03:43,929 INFO __main__: Collected 63 feature samples for point 3
2025-12-07 23:03:43,929 INFO __main__: Point 4 / 9: (384, 832) — look at the yellow dot.
2025-12-07 23:03:44,743 INFO __main__: Collecting samples for point: (384, 832)
2025-12-07 23:03:46,792 INFO __main__: Collected 61 feature samples for point 4
2025-12-07 23:03:46,792 INFO __main__: Point 5 / 9: (1280, 832) — look at the yellow dot.
2025-12-07 23:03:47,601 INFO __main__: Collecting samples for point: (1280, 832)
2025-12-07 23:03:49,648 INFO __main__: Collected 30 feature samples for point 5
2025-12-07 23:03:49,649 INFO __main__: Point 6 / 9: (2176, 832) — look at the yellow dot.
2025-12-07 23:03:50,467 INFO __main__: Collecting samples for point: (2176, 832)
2025-12-07 23:03:52,514 INFO __main__: Collected 31 feature samples for point 6
2025-12-07 23:03:52,514 INFO __main__: Point 7 / 9: (384, 1414) — look at the yellow dot.
2025-12-07 23:03:53,325 INFO __main__: Collecting samples for point: (384, 1414)
2025-12-07 23:03:55,343 INFO __main__: Collected 57 feature samples for point 7
2025-12-07 23:03:55,344 INFO __main__: Point 8 / 9: (1280, 1414) — look at the yellow dot.
2025-12-07 23:03:56,157 INFO __main__: Collecting samples for point: (1280, 1414)
2025-12-07 23:03:58,157 INFO __main__: Collected 60 feature samples for point 8
2025-12-07 23:03:58,158 INFO __main__: Point 9 / 9: (2176, 1414) — look at the yellow dot.
2025-12-07 23:03:58,964 INFO __main__: Collecting samples for point: (2176, 1414)
2025-12-07 23:04:01,008 INFO __main__: Collected 62 feature samples for point 9
2025-12-07 23:04:01,009 INFO __main__: Fitting calibration matrix on 468 samples
```

```
2025-12-07 23:04:01,013 INFO __main__: Calibration done. Saved gaze_calib.npz with M=[[ -9259.412   -1389.3512]
 [-18507.611  -40196.684 ]
 [ 13656.457   18392.57  ]]
```

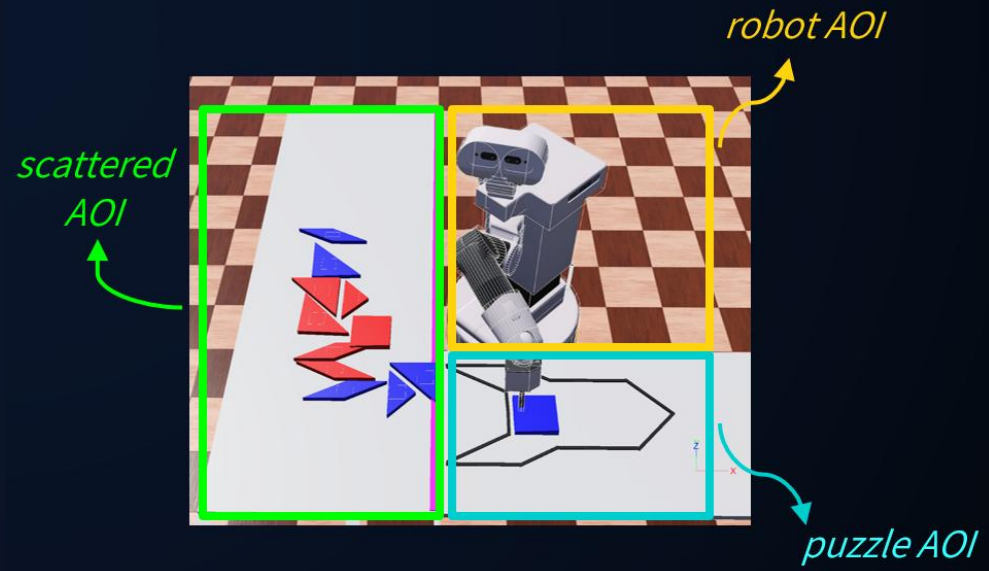
Gaze Server Features

Component	Purpose
MediaPipe Face Mesh	Real-time facial landmark extraction (eyes, pupils region).
OpenCV	Frame capture, preprocessing, drawing overlays. Imported cv2 for gaze calibration window and detection in the final run.
gazeutils.FrameSample	Internal wrapper that standardizes gaze data (AOI label, blink flag, timestamp).
FastAPI	Provides a REST server that exposes gaze metrics and decisions to the robot game logic. It handles the exception cases where HTTP server doesn't work as well.
Uvicorn	ASGI server that runs the FastAPI gaze service.
NumPy	Vector math for dwell computation and scoring.

before going to our EVALUATION METRICS....

AOI

1. Robot (-ve)
2. Puzzle (+ve)
3. Scattered (-ve)
4. Outside (neutral)



Gaze Window

Time: 5 seconds

TERMINOLOGY

Blinks

⇒ Threshold: 3
⇒ **Less Blinks:** calm/non-anxious OR approval
⇒ **More Blinks:** discomfort/uncertainity OR disapproval

Dwell Time

⇒ Equal △t to each AOI
⇒ Based on sentiment of each AOI that leads in time

Gaze Shifts

⇒ Threshold: 4
⇒ AOI targeted: robot & puzzle
⇒ Transitions among AOIs
⇒ stable gaze **VS** frequent behaviour

The Ratio

⇒ Robot / Puzzle Ratio
⇒ Derived from dwell times
⇒ If num>den then NEGATIVE
⇒ If den>num then POSITIVE
⇒ If den == 0 then NEUTRAL
⇒ If num == 0 then POSITIVE

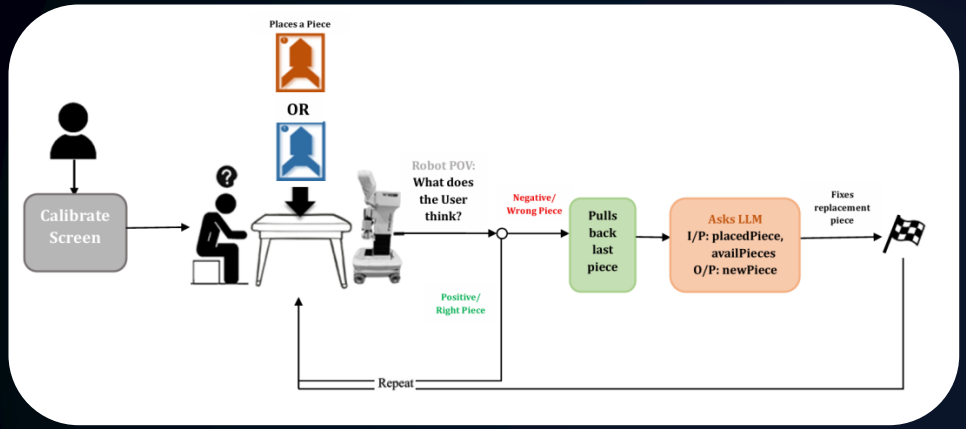
NOW for the EVALUATION METRICS!

Metric	Code Variable	Threshold / Logic	Score	Weight
Blink Count	`blink_count`	≤3 positive, >3 negative	`blink_score`	**0.2**
AOI Shifts	`shifts`	≤4 positive, >4 negative	`shift_score`	**0.4**
Dwell Balance	`dwell[...]`	(Puzzle+Outside) vs (Robot+Scattered)	`dwell_score`	**0.3**
Robot/Puzzle Ratio	`ratio_score`	Puzzle>Robot → +1; Robot>Puzzle → −1	`ratio_score`	**0.1**

***FINAL SCORE** = W BLINK * blink score + WSHIFT * shift score + W DWELL * dwell score +W RATIO * ratio score*

***Decision Rule:** if final score >= 0: return "Positive" else: return "Negative"*

LLM INTEGRATION



- ⇒ We're currently using ***GPT-4o-mini***
- ⇒ Whenever there is negative user feedback detected – pulls the previous piece back
- ⇒ Calls LLM with placedPiece, availPieces and a well-defined prompt explaining the setup
- ⇒ We get the name of the piece that the LLM suggests as the result for replacement
- ⇒ Result as JSON

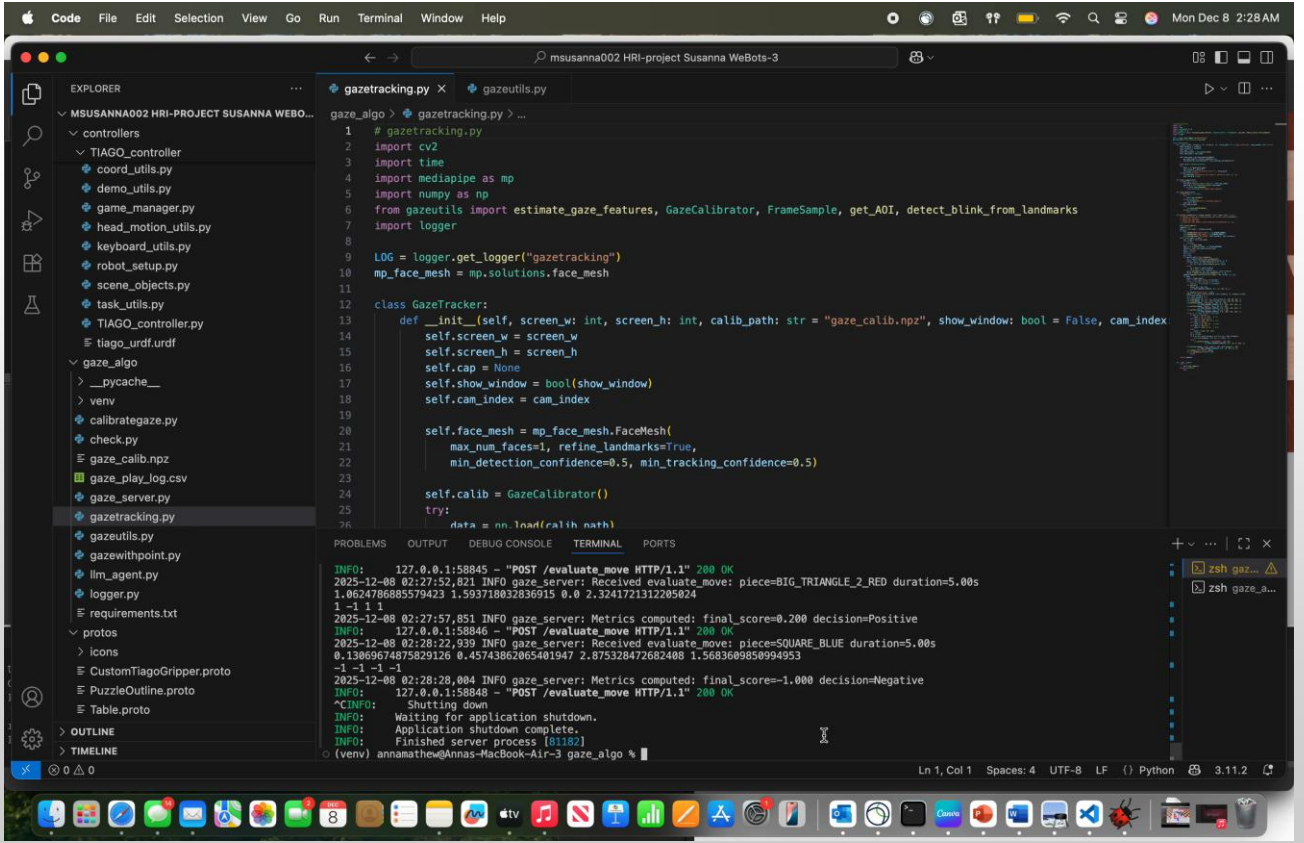
"WHY is our LLM SOO DUMB?!!"
- Anna Mathew

04

RESULTS

ANALYSIS & LESSONS

A SAMPLE PUZZLE DEMO



Results & Explanation

1. **PLAY 1**
   ROBOT KEEPS: parallelogram_red2
   USER'S ACTIONS: looked at the puzzle AOI
   FEEDBACK TAKEN: positive
   OUTPUT: next move

2. **PLAY 2**
   ROBOT KEEPS: big_triangle_red2
   USER'S ACTIONS: looked at Puzzle AOI, no blinks
   FEEDBACK TAKEN: positive
   OUTPUT: next move

3. **PLAY 3**
   ROBOT KEEPS: parallelogram_red1
   USER'S ACTIONS: looked outside, neutral blink rate
   FEEDBACK TAKEN: positive
   OUTPUT: next move

4. **PLAY 4**
   ROBOT KEEPS: small_triangle_blue
   USER'S ACTIONS: shifted gaze frequently b/w robot-puzzle AOI
   FEEDBACK TAKEN: negative
   OUTPUT: removed & LLM suggested big_triangle_red1

4. **PLAY 4**
   ROBOT KEEPS: small_triangle_blue
   USER'S ACTIONS: shifted gaze frequently b/w robot-puzzle AOI
   FEEDBACK TAKEN: negative
   OUTPUT: removed & LLM suggested big_triangle_red1

LLM's Piece: big_triangle_red1
FEEDBACK: positive
OUTPUT: next move

5. **PLAY 5**
   ROBOT KEEPS: square_blue
   USER'S ACTIONS: random shifts, blinks more
   FEEDBACK TAKEN: negative
   OUTPUT: removed & LLM suggested square_red

LLM's Piece: square_red
FEEDBACK: positive
OUTPUT: next move

6. **PLAY 6**
   ROBOT KEEPS: small_traiangle_red
   USER'S ACTIONS: 1 shift, hold on puzzle AOI
   FEEDBACK TAKEN: positive
   OUTPUT: next move

Evaluating our System

1. Correction Success Rate

- How often the gaze server was right in predicting the user's feedback
- Heavily subjective from user to user
- Gaze was 100% accurate in some users but was significantly lower in some others (5 user cases)

2. LLM-Assisted Piece Selection Accuracy

- How often the LLM chose the next Tangram piece that's the right fit
- LLM is not getting smarter as every call is a new call
- Out of our sample runs 30% accuracy of the LLM was detected

3. Over-Corrections (False Positives) & Missed Corrections (False Negatives)

- Over-corrections: No. of times robots misinterprets neutral human behavior as a negative cue
- Missed Corrections: Robot fails to react to human cue when there is one
- Over Correctness was seen with a distracted user and very fidgety one
- No Missed Corrections as far as environmental and lighting factors don't interfere

4. Correction latency Time from human cue → robot response start

- Time from human cue ⇒ robot response time
- Not more than 1 sec delay after gaze window time for robot response

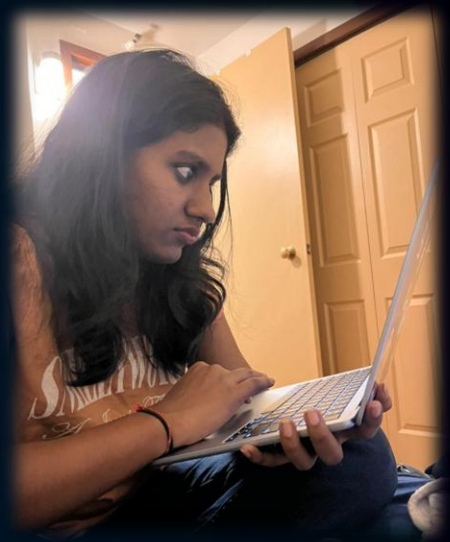
Robot in front of our robot

RESULT: failed calibration

Lights out!

RESULT: runs calibration but failed detection

Worst user: the STONE



RESULT: all positives

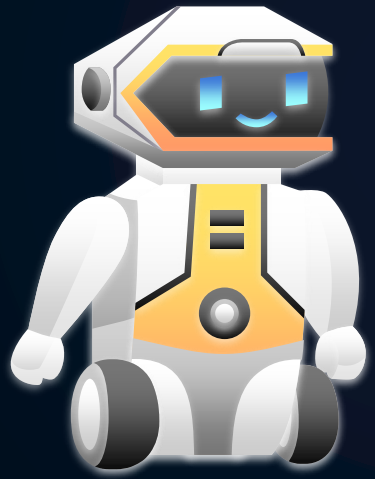
Bloopers? EDGE CASES & SETBACKS!

Lessons

- Very dependent on the user since positives and negatives alter from person to person
- Gaze on webcams are very sensitive, calibration needed more than once
- Learnt WeBot setups!
- Environmental factors play a huge part
- Ethics and safety to be considered in this nature of project

Future Work

- Improve with iris-tracking models
- Include more concrete metrics
- Analyze facial expressions, head poses, hand pointing to increase accuracy in feedback
- Better ways to communicate with the LLM – I/P sent to it
- Maybe more fun puzzles?



Thank You!

Happy Holidays!

Credits: This presentation template was created by **Slidesgo**, and includes icons by **Flaticon** and infographics & images by **Freepik**